Astana IT University
School of Software Engineering

Diploma Project

Development of a System for Detecting and Anonymizing Personal Data Before Sending Text to Artificial Intelligence Services

Technology Comparison

Authors: Dossan Amina, Abzhamal Nazerke
Educational programme: Software Engineering
Supervisor: Sakhipov A.A., PhD,
Assistant Professor

Astana, 2026

Contents

1 Introduction **3**

2 Requirements Used for the Decisions **3**

3 Evaluation Method **3**

4 Decision 1: Server Platform **4**

5 Decision 2: Data Storage **4**

5.1 Main Database 4

5.2 Pseudonym Mapping Store 4

6 Decision 3: Personal Data Detection Method **5**

7 Decision 4: Named Entity Recognition Model **6**

8 Decision 5: Component Interaction **6**

9 Decision 6: Client Type **6**

10 Summary of the Selected Technology Stack **8**

1. Introduction

This document justifies the key technology decisions of the project. For each decision, two or three reasonable alternatives are compared against criteria derived from the project requirements, and the evaluation is supported by technical documentation and by evidence from the literature. Six decisions are considered: the server platform, data storage (main database and pseudonym mapping store), the personal data detection method, the named entity recognition (NER) model, the method of component interaction, and the client type.

2. Requirements Used for the Decisions

The decisions are based on the key requirements listed in Table 1. The complete specification is given in the document *Requirements Specification*.

Table 1: Key requirements used for the technology decisions

ID	Requirement	Acceptance criterion
FR-1	Detect personal data (names, IIN/BIN, phone number, e-mail, address, card number, IBAN) in Kazakh, Russian, English and mixed text	Recall ≥ 0.95 for structured identifiers on the test corpus
FR-2	Validate IIN/BIN and card numbers by checksum	No false detections on 100 invalid synthetic numbers
FR-3	Replace detected data with pseudonyms; the same entity receives the same pseudonym within a text	100% consistency on test texts with repeated entities
FR-4	Restore the original values in the response of the AI service	At least 95% of responses fully restored
FR-5	Intercept text before it is sent to ChatGPT or Gemini and let the user confirm the detected data	Works in the web interfaces of both services
NFR-1	Low added latency	At most 1 s for a text of up to 2,000 characters
NFR-2	Original data and the pseudonym mapping never leave the organization; the mapping is encrypted and deleted automatically	AES encryption; retention of at most 15 min; no original values in logs
NFR-3	On-premise deployment	Started with a single Docker Compose command

3. Evaluation Method

Each alternative is rated against each criterion on a scale from 1 (poor) to 5 (excellent). Each criterion has a weight that reflects the priority of the related requirement: 3 for a mandatory requirement, 2 for a desirable one, and 1 for an additional consideration. The total score of an alternative is the sum of the products of weights and ratings. In the tables below, the weight is given in parentheses after each criterion, and the rating is followed by a short justification.

4. Decision 1: Server Platform

Table 2: Comparison of server platforms

Criterion (weight)	FastAPI (Python)	Express (Node.js)	Django REST Framework (Python)
Native use of Presidio, spaCy and Transformers, FR-1 (3)	5 — same language	2 — requires a separate Python service and an extra network call	5 — same language
Asynchronous input/output and streaming of responses, FR-4, NFR-1 (3)	5 — asynchronous by design; streaming responses [1]	5 — event loop, native streams	3 — asynchronous views require ASGI; ORM transactions are not asynchronous [2]
Automatic API documentation (2)	5 — OpenAPI documentation generated from code [1]	3 — requires additional packages	4 — through extensions
Lightweight framework for a gateway service (1)	5	5	3 — full-stack framework
Team experience (1)	3	5 — previous projects	3
Weighted total	**48**	**37**	**38**

FastAPI is selected (Table 2). The detection core depends on Python libraries (FR-1), and FastAPI executes them in the same process, supports streaming of language model responses (FR-4) and generates OpenAPI documentation automatically. A Python server is also used by the closest middleware prototype described in the literature [3]. The team has more experience with Express; this disadvantage is accepted because Express would require two runtime environments and an additional internal call for each request, which conflicts with NFR-1.

5. Decision 2: Data Storage

5.1. Main Database

The main database stores users, roles, policies and detection logs. The alternatives are compared in Table 3.

PostgreSQL is selected because the data are structured and the administration panel requires aggregated reports. Its schema is less flexible than that of MongoDB; policies with variable fields can be stored in JSONB columns.

5.2. Pseudonym Mapping Store

The mapping store keeps the correspondence between original values and pseudonyms for the duration of a request. The alternatives are compared in Table 4.

Redis is selected, with an expiry time for each key and values encrypted with AES by the application before storage. Limited retention of personal data is also expected by users [3]. The cost of this decision is one additional service in the deployment.

Table 3: Comparison of main database options

Criterion (weight)	PostgreSQL	MongoDB	SQLite
Structured, related data (users, roles, policies, logs) (3)	5 — relational schema with constraints	3 — documents; relations handled in code	4 — relational
Reporting on logs for the administration panel (2)	5 — SQL aggregation	3 — aggregation pipeline	4 — SQL
Concurrent access from several workers, NFR-1 (2)	5	5	2 — single writer, file-based
Docker and on-premise deployment, NFR-3 (2)	5	5	5
Team experience (1)	4	5	3
Weighted total	**49**	**40**	**37**

Table 4: Comparison of pseudonym mapping store options

Criterion (weight)	Redis	PostgreSQL table	In-process memory
Automatic deletion after expiry, NFR-2 (3)	5 — keys are deleted automatically after the timeout [4]	2 — requires a scheduled cleanup job	3 — custom timer; lost on restart
Fast reading and writing on every request, NFR-1 (3)	5 — in-memory	3 — disk-based	5
Shared between several workers (2)	5	5	1 — each worker has its own memory
Data not stored longer than necessary, NFR-2 (3)	4 — persistence can be disabled	2 — stored on disk and in backups	5
Weighted total	**52**	**31**	**41**

6. Decision 3: Personal Data Detection Method

The literature supports the ratings in Table 5. Regular expressions without checksum validation reached only 7.5% precision for IBAN codes [9], whereas validated rules are used successfully in recent middleware [8]. XLM-R is the best model on the Kazakh NER dataset [5] and on Korean personal data (F1 of 0.89) [10], but NER alone performs worst on language-specific identifiers [10]. Detection by a large language model is the most accurate in a recent comparison [6]; however, pseudonymization performed by language models leaked 35.5% of personal data compared with 10.2% for an NER-based approach [11], and the best results require sending data to the provider. All three recent systems for the protection of prompts combine rules with a neural model [7, 3, 8].

The hybrid approach is therefore selected: validated pattern recognizers for structured identifiers (IIN/BIN with checksum, phone numbers in Kazakhstan formats, card numbers with the Luhn algorithm, Kazakhstan IBAN), an XLM-R-based NER model for names, locations and organizations, and a gazetteer of Kazakh names, since names of non-European origin are missed more often by standard tools [6]. Presidio is used as the framework for the recognizers and as the evaluation baseline. Two components have to be maintained, and quasi-identifiers and attribute inference remain outside the scope of the project.

Table 5: Comparison of personal data detection methods

Criterion (weight)	Rules only (patterns and checksums)	NER only (spaCy or XLM-R)	Detection by a large language model	Hybrid: rules and NER
Structured identifiers (IIN/BIN, phone, card, IBAN), FR-1, FR-2 (3)	5 — exact patterns with checksums	1 — IIN was never typed correctly in the Presidio experiment	3 — no checksum validation	5
Names, locations and organizations in Kazakh, Russian and English, FR-1 (3)	1 — cannot generalize	4 — XLM-R F1 of 97.22% on Kazakh NER [5]	4 — high recall (0.96) [6], but requires fine-tuning and an external service	4
Latency, NFR-1 (3)	5 — 0.1–10 ms [7, 8]	4 — encoder model 0.15–0.2 s [7, 8]	1 — local generative model 2–3.2 s per prompt [7, 8]	4
Data remain within the organization, NFR-2 (3)	5	5	2 — external service or a large local model (about 6 GB of GPU memory [7])	5
Weighted total	**48**	**42**	**30**	**54**

7. Decision 4: Named Entity Recognition Model

XLM-RoBERTa-base, fine-tuned on KazNERD and Russian NER data, is selected (Table 6); multilingual BERT is kept as a fallback in case the latency target is not met. XLM-R is also the backbone of the closest middleware prototype [3] and the best model family on KDPII [10]. The model requires more memory than the spaCy models, and its advantage over multilingual BERT is small, so both will be measured on the project test corpus.

8. Decision 5: Component Interaction

REST over HTTPS with JSON is selected, with streaming for language model responses (Table 7). Its payloads are larger than those of gRPC, which is negligible for text prompts.

9. Decision 6: Client Type

A browser extension (Manifest V3) is selected, complemented by an API proxy as a second entry point for programmatic use (Table 8). Unlike Casper [7], the NER model runs on the organization's server, so the extension remains lightweight and requires no GPU on user machines. Changes in the interfaces of AI services may break interception, and Casper reports limitations of the browser interface when intercepting submission; therefore, support is limited to two services, and the proxy remains a stable alternative.

Table 6: Comparison of NER models

Criterion (weight)	XLM-RoBERTa-base, fine-tuned on KazNERD and Russian data	Multilingual BERT, fine-tuned	spaCy models (Russian and multilingual)
Quality on Kazakh, FR-1 (3)	5 — F1 of 97.22% on KazNERD [5]	4 — F1 of 96.24% on KazNERD [5]	2 — correct-type recall of 0.71 in the Presidio experiment
Quality on Russian, FR-1 (3)	4 — strong multilingual model; best model family on KDPII [10]	4	3 — recall of 0.80 in the Presidio experiment
Code-switched text, FR-1 (2)	5 — pretrained on web text in 100 languages, including 6.4 GiB of Kazakh [12]	3 — pretrained on Wikipedia only	2 — separate model for each language
Latency on CPU, NFR-1 (3)	4 — encoder, single forward pass (about 0.15 s [8])	4	5
Model size and resources (2)	3 — about 280 million parameters	4 — about 180 million parameters	5
Integration with Presidio (2)	4 — through the Transformers NLP engine	4 — same	5 — native
Weighted total	**63**	**58**	**54**

Table 7: Comparison of component interaction methods

Criterion (weight)	REST over HTTP with JSON (streaming via chunked responses or SSE)	gRPC	Message queue (e.g. RabbitMQ)
Callable from a browser extension, FR-5 (3)	5 — native `fetch`	2 — browsers require gRPC-Web and a proxy [13]	1 — not callable from a browser directly
Synchronous request and response with low latency, NFR-1 (3)	4	5	2 — asynchronous by design
Compatibility with provider interfaces (OpenAI and Gemini use HTTP and JSON), FR-4 (3)	5	2 — requires a translation layer	1
Simplicity and documentation (2)	5	3	2
Weighted total	**52**	**33**	**16**

Table 8: Comparison of client types

Criterion (weight)	Browser extension	Separate web application	API proxy only
Works where users already use ChatGPT or Gemini, FR-5 (3)	5 — content scripts can read and modify web pages [14]; feasibility shown by Casper [7]	2 — the user copies text to another site (the PromptShield model [3])	1 — only for programmatic use, not for web interfaces
User can review detected data before sending, FR-5 (3)	5	5	1
Works for internal applications that use language model APIs (2)	2	2	5
Robustness to changes in the ChatGPT or Gemini interface (2)	2 — page selectors may break	5	5
Weighted total	**38**	**35**	**26**

10. Summary of the Selected Technology Stack

Table 9: Selected technology stack

Layer	Choice	Main requirements
Server	Python, FastAPI	FR-1, FR-4, NFR-1
Detection	Presidio framework, custom Kazakhstan recognizers with checksums, XLM-R NER model, Kazakh name gazetteer	FR-1, FR-2
Main database	PostgreSQL	Administration panel, logs
Mapping store	Redis with expiry and AES encryption	NFR-2
Interaction	REST over HTTPS with JSON, streaming	FR-4, FR-5
Client	Browser extension (Manifest V3) and API proxy	FR-5
Deployment	Docker Compose	NFR-3

References

[1] FastAPI, “Custom response — HTML, stream, file, others,” documentation. [Online]. Available: https://fastapi.tiangolo.com/advanced/custom-response/

[2] Django Software Foundation, “Asynchronous support,” Django documentation. [Online]. Available: https://docs.djangoproject.com/en/stable/topics/async/

[3] A. I. Asimine, F. Salhab Doria Ricardo, F. Torgo, G. Choudhary, and N. Dragoni, “PromptShield: Protecting user privacy in AI chatbots,” *Software*, vol. 5, no. 2, art. 24, 2026, doi: 10.3390/software5020024.

[4] Redis, “EXPIRE,” command reference. [Online]. Available: https://redis.io/docs/latest/commands/expire/

[5] R. Yeshpanov, Y. Khassanov, and H. A. Varol, "KazNERD: Kazakh named entity recognition dataset," in *Proc. LREC*, 2022, pp. 417–426. [Online]. Available: https://aclanthology.org/2022.lrec-1.44

[6] Z. Ji, K. R. Koedinger, Y. Shen, and J. Lin, "Enhancing the de-identification of personally identifiable information in educational data," *Journal of Educational Data Mining*, vol. 17, no. 2, pp. 55–85, 2025. [Online]. Available: https://jedm.educationaldatamining.org/index.php/JEDM/article/view/936

[7] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt sanitization for protecting user privacy in web-based large language models," in *Proc. IEEE 12th Int. Conf. on Cyber Security and Cloud Computing (CSCloud)*, 2025, pp. 122–131, doi: 10.1109/CSCloud66326.2025.00027.

[8] A. A. Hsine and A. Arabo, "A privacy-preserving middleware architecture for detecting prompt injection and sensitive data exposure in large-language-model interactions," *Electronics*, vol. 15, no. 16, art. 3554, 2026, doi: 10.3390/electronics15163554.

[9] N. Subramani, S. Luccioni, J. Dodge, and M. Mitchell, "Detecting personal information in training corpora: an analysis," in *Proc. 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP)*, 2023, pp. 208–220, doi: 10.18653/v1/2023.trustnlp-1.18.

[10] L. Fei, Y. Kang, S. Park, Y. Jang, J. Lee, and H. Kim, "KDPII: A new Korean dialogic dataset for the deidentification of personally identifiable information," *IEEE Access*, vol. 12, pp. 135626–135641, 2024, doi: 10.1109/ACCESS.2024.3461804.

[11] O. Yermilov, V. Raheja, and A. Chernodub, "Privacy- and utility-preserving NLP with anonymized data: A case study of pseudonymization," in *Proc. 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP)*, 2023, doi: 10.18653/v1/2023.trustnlp-1.20.

[12] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proc. ACL*, 2020, pp. 8440–8451, doi: 10.18653/v1/2020.acl-main.747.

[13] gRPC Authors, "gRPC-Web basics tutorial." [Online]. Available: https://grpc.io/docs/platforms/web/basics/

[14] Google, "Content scripts," Chrome Extensions documentation. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts